SQ-HotRoll: Session-Quiescence-Gated Progressive Schema Evolution with Inline Complement Lenses for Stateful Edge and Relational Databases

Augusto Cerrutti

Independent Systems Researcher

augustocerruttisatto@gmail.com · ORCID: 0009-0005-7279-8319 · github.com/kanchingyg

Repository: github.com/kanchingyg/sql-hotroll · Preprint DOI: 10.5281/zenodo.23675281 · Archive DOI: 10.5281/zenodo.23675683

October 27, 2026

Abstract

Web applications increasingly pair edge Workers and regional points of presence (PoPs) with serverless SQLite engines (Cloudflare D1) or pooled PostgreSQL databases while serving long-lived browser single-page applications (SPAs), desktop clients, and native database drivers. Existing online schema change protocols execute either a blocking table rebuild or a global atomic cutover that switches all database connections from schema v_1 to v_2 at a single instant. Global cutovers break un-reloaded client sessions whose active bundles still emit v_1 payloads, while naive dual-write triggers suffer from sticky recursion flags, multi-table join billing amplification on row-metered engines, and lost updates across shared records. This paper presents **SQ-HotRoll** (`sql-hotroll`) [10], a progressive schema evolution architecture that decouples physical dual-table readiness from per-session logical schema visibility. SQ-HotRoll synchronizes v_1 and v_2 tables using bidirectional relational lenses with an inline JSON complement column (`_lc`) and a monotonic row-version guard (`_row_ver`), routes additive or computed columns through SQLite's zero-copy `VIRTUAL` generated column fast-path (1 row read per query and 1 row written per mutation with 0% disk overhead), and advances active client sessions only at stream-aware quiescence boundaries (top-level `HTTP 200` navigations, 90-second AFK idle windows guarded by an input `AbortController` with zero active streams or multipart leases, or explicit transaction checkpoints) using a portable signed epoch envelope (`X-SQ-Epoch-Token`). Across 960 measured local engine runs (30 seeded runs per cell after 5 warmup runs, isolating in-process SQLite WAL and local PostgreSQL 17.11 engine execution overhead from client-to-PoP WAN round-trip latency), SQ-HotRoll reduces in-process SQLite WAL $p_{99.9}$ tail latency by 49.9× (1,450 μs / 1.45 ms to 28–29 μs / 0.029 ms) and local PostgreSQL 17.11 $p_{99.9}$ tail latency by 64.5× (9.03 ms to 0.14 ms). It achieves 0 schema-mismatch errors (0/1,200 requests per run, vs. 486/1,200 under uncoordinated global cutover, Table 2), keeps structural D1 read cost at 1 row read per `SELECT` and active migration write cost at 2 rows written per mutation reverting to 1 after garbage collection (Figure 2), shrinks peak backfill WAL size by 55.70×, and executes single-flag rollbacks in 17 μs (0.017 ms) on in-process SQLite WAL and 50 μs (0.05 ms) on PostgreSQL 17.11. Polyglot native driver validation across C, Rust, PHP, and TypeScript confirms 40/40 clean statement executions (0 plan-cache crashes), while multi-PoP CDN `HTTP 206` byte-range gating eliminates the 488/1,000 (48.8%) cross-epoch slice corruption of unpinned range caching. All nine safety, causal monotonicity, stream-aware quiescence, and lossless rollback theorems are mechanically verified in Microsoft Z3 ($\sim$ 36 ms total across all 9 SMT obligations, 0.7–21 ms per theorem) alongside an exhaustive Spin 6.5.2 LTL model check (549 states, 983 transitions, 0 errors).

1 Introduction

Stateful web applications increasingly pair edge Workers and regional points of presence (PoPs) with row-metered serverless SQLite engines such as Cloudflare D1 [20, 33] or connection-pooled PostgreSQL databases [13, 31, 28]. Across these deployments, browser single-page applications (SPAs), desktop applications, and mobile in-app WebViews hold uncommitted form state, active byte-range downloads, optimistic mutations, and schema-coupled serializers in memory across hours or days [22, 25].

Database schema evolution systems developed over the past two decades (including Google F1's asynchronous two-epoch protocol [29], PRISM/PRISM++ [15, 14], InVerDa [21], BullFrog [3], snapshot-isolated DDL engines [24]) treat schema migration as a purely backend storage transition. Once shadow-table backfill finishes, production tools such as `gh-ost`, `pgroll`, and `reshape` execute a global atomic cutover by renaming tables or flipping a database `search_path` pointer across all incoming connections at once.

While global atomic cutover avoids long `ACCESS EXCLUSIVE` table locks, it introduces a severe frontend-backend synchronization hazard termed *in-flight session schema skew*. A client session opened ten minutes before cutover continues to issue asynchronous mutations formatted for schema v_1; the moment the backend globally flips to v_2, those un-reloaded sessions fail with HTTP 500 column or constraint errors (486/1,200 requests per

run, or 40.5% on D1/SQLite and 40.83% on PostgreSQL 17.11 in Table 2). Attempting to serve both v_1 and v_2 simultaneously via naive dual-write triggers and separate complement tables introduces three production failure modes: (1) on row-metered engines like Cloudflare D1, a 3-way `LEFT JOIN` across v_2, v_1, and an external complement table inflates read billing to 2.50 rows/read and write billing to 3.87 rows/write (Figure 2b); (2) boolean recursion flags (`_sync_src = 1`) inside SQLite `AFTER UPDATE` triggers remain sticky on disk, silently dropping subsequent alternating updates ($v_2 \to v_1 \to v_2$) on hot keys (13.6 lost updates per run); and (3) storing schema epoch state in origin-bound browser cookies breaks under multi-tab concurrency, Anycast PoP failover during `HTTP 206` byte-range downloads, and pooled native database drivers whose cached prepared statements crash with `SQLSTATE 0A000` (`cached plan must not change result type`) or `SQLITE_SCHEMA (17)` on DDL/view swaps.

Central Hypothesis. This work formulates and tests a falsifiable systems proposition: *decoupling physical storage synchronization (handled globally via bidirectional lenses, monotonic `_row_ver` guards, and inline `_lc` complements) from logical schema visibility (promoted per client or cohort strictly at verified stream-aware quiescence boundaries via stateless signed epoch envelopes) eliminates both DDL lock stalls and mid-session schema-skew HTTP 500s without multi-table read-join amplification.*

This paper presents the design and implementation of **SQ-HotRoll** (`sql-hotroll`) [10], a session-quiescence-gated progressive schema evolution engine for Cloudflare Workers/D1, `bun:sqlite`, and PostgreSQL 17.11. The database maintains bijectively synchronized v_1 and v_2 tables via inline complement lenses [4, 19, 17], while each client session advances from v_1 to v_2 only when it reaches a verified quiescence boundary [22, 25]. Specifically, this paper makes four contributions:

1. **Bijective Inline Complement Lenses and Monotonic Triggers (Section 3):** By embedding a JSON complement column (`_lc`) and a monotonic version guard (`_row_ver`) directly into T_{v_1} and T_{v_2}, SQ-HotRoll achieves 1 row read per `SELECT` (after phase-gated keyset backfill) and 2 rows written per mutation during the active migration window (reverting to 1 after garbage collection) with 0 lost updates and sub-millisecond single-flag rollback (17 μs / 0.017 ms on in-process SQLite WAL and 50 μs / 0.05 ms on PostgreSQL 17.11).
2. **Stream-Aware Session Quiescence and Multi-PoP CDN Byte-Range Safety (Section 4):** A portable epoch envelope (`X-SQ-Epoch-Token`) and stream-aware quiescence gate ($\mathcal{Q}_{\text{any}}$ requiring $S_{\text{streams}} = 0$ and $L_{\text{multipart}} = \emptyset$) advance browser SPAs, desktop clients, and mobile WebViews across multi-CDN edges without cross-tab cookie clobbering, while an `ETag` + `If-Range` + `X-SQ-Stream-Lease` CDN gate eliminates cross-epoch `HTTP 206` byte-slice corruption (0/1,000 vs. 488/1,000 under unpinned range caching).
3. **3-Tier DDL AST Classifier and Polyglot Plan-Cache Isolation (Section 4):** A 3-Tier DDL AST Classifier routes additive and computed projections through SQLite's zero-copy `VIRTUAL` generated column fast-path (Tier 0: 1 row read per query, 1 row written per mutation, 0% disk overhead, 0.446 ms setup, 206.9× faster than shadow-table copy), routes shared-table column additions through Tier 1 (1 row read, 2 rows written during migration), and reserves Tier 2 shadow tables for structural rewrites, backed by an adaptive WAL-paced keyset backfiller (55.70× smaller peak `.sqlite-wal`) and polyglot AST table-alias rewriting (40/40 clean executions across C, Rust, PHP, and TypeScript with 0 plan-cache crashes).
4. **Dual-Method Formal Verification and Empirical Evaluation (Sections 3, 5):** Nine safety theorems are mechanically verified in Microsoft Z3 [16] (∼ 36 ms total across all 9 SMT obligations, 0.7–21 ms per theorem) and Spin 6.5.2 [23] (549 states, 0 errors), and SQ-HotRoll is evaluated across 960 measured local engine runs on YCSB [12], TPC-C, SPA traces, and polyglot/CDN testbeds.

Section 2 defines the system model, failure taxonomy, and related work. Section 3 presents the inline complement lens model and Z3/Spin proofs. Section 4 details the stream-aware quiescence protocol, multi-PoP CDN byte-range gate, polyglot SQL compilation, and CLI provenance gate. Section 5 reports the empirical evaluation and threats to validity, and Section 6 discusses boundaries and concludes.

2 System Model, Failure Taxonomy, and Related Work

2.1 Edge-Relational System Model

Consider a stateful web application backed by either a single-writer SQLite WAL database exposed through stateless HTTP/RPC worker isolates (Cloudflare D1 [20]) or a connection-pooled relational cluster (PostgreSQL 17.11 behind Cloudflare Hyperdrive or PgBouncer). Let T_{v_1} denote the initial table schema at epoch v_1 and T_{v_2} denote the target schema at epoch v_2. A fleet of stateful client sessions $\mathcal{S} = \{s_1, \ldots, s_M\}$ interacts with the backend over time. Each session s_k holds:

- an active client code bundle locked to schema epoch $e(s_k) \in \{v_1, v_2\}$;
- a physical causal replication watermark $\beta(s_k)$ (a D1 WAL commit bookmark `x-d1-bookmark` or PostgreSQL LSN `pg_current_wal_lsn()`) [34];
- an in-flight RPC counter $F(s_k) \in \mathbb{N}_0$, a dirty unsubmitted form-input counter $D(s_k) \in \mathbb{N}_0$, an active

byte-range stream count $S_{\text{streams}}(s_k) \in \mathbb{N}_0$, and an active multipart upload lease set $L_{\text{multipart}}(s_k)$.

2.2 Five Production Failure Modes in Naive Dual-Table Migrations

Empirical profiling on Cloudflare D1/SQLite and PostgreSQL 17.11 using companion diagnostic tools (`pg-lock-xray` [8], `dbwatch` [6], `wrangler-doctor` [11], and `flaredump` [7]) reveals five distinct production hazards when attempting zero-downtime migrations without session-quiescence coordination:

- **Trap 1 (3-Way Read Join and D1 Billing Amplification):** Storing lens complements in a separate third table `_lens_complement` requires every v_2 read before backfill completion to execute a 3-way `LEFT JOIN` across T_{v_2}, T_{v_1}, and `_lens_complement`. On Cloudflare D1, which bills per B-tree row scanned and written, this inflates read cost to 2.50 rows/query and write cost to 3.87 rows/mutation (Figure 2b).
- **Trap 2 (Sticky `_sync_src` Trigger Recursion Bug):** Because SQLite lacks PostgreSQL's connection-local `pg_trigger_depth()`, naive dual-write triggers guard against infinite loops using a boolean column `_sync_src = 1`. Because SQLite forbids a trigger from clearing `_sync_src = 0` after its peer trigger finishes without firing another `UPDATE`, the flag remains 1 on disk; a subsequent write on the opposite version sees `_sync_src = 1` and silently skips propagation (13.6 lost writes per run).
- **Trap 3 (Prepared-Statement Plan-Cache Crash under Connection Pooling):** When transaction-mode poolers (Cloudflare Hyperdrive or PgBouncer) or native C/Rust/PHP drivers reuse prepared statements across a DDL `ALTER TABLE` or view swap, PostgreSQL raises `SQLSTATE 0A000` (`cached plan must not change result type`) on the 6th execution (when `plancache.c` transitions from 5 custom plans to a generic cached plan) and SQLite raises `SQLITE_SCHEMA (17)`.
- **Trap 4 (Multi-Tab Cookie Clobbering and Anycast CDN Byte-Range Tearing):** Storing the active schema epoch in an origin-bound browser cookie causes a newly opened Tab B (promoted to v_2) to overwrite the shared cookie jar, routing an older Tab A's in-flight v_1 XHR calls to v_2 and causing immediate HTTP 500 errors. Similarly, when a large `HTTP 206 Partial Content` download or resumable upload spans an Anycast PoP failover during a schema transition, unpinned CDN range caches concatenate byte slices from epoch v_1 and epoch v_2, silently corrupting multi-chunk payloads (488/1,000 corrupted downloads in Section 5).
- **Trap 5 (Hardware/POS and Native Host-Bridge Desynchronization):** Non-browser clients (warehouse POS scanners pinned by MAC OUI or mTLS certificates, desktop applications, and mobile `WKWebView` mini-apps) do not send `Sec-Fetch-Mode: navigate` headers; promoting an embedded WebView to v_2 before the host native Objective-C/Kotlin bridge upgrades its protobuf schema causes host-bridge deserialization crashes.

2.3 Related Work

Online Schema Evolution and Cutover Protocols. Google F1 [29] formalized asynchronous two-epoch schema evolution (*absent* → *delete-only* → *write-only* → *public*) so distributed SQL query routers never hold more than two adjacent schema leases, inspiring CockroachDB [31] and Spanner [13]. Curino et al. [15, 14] developed PRISM and PRISM++ to automate query and update rewriting across schema versions, while Bhattacherjee et al. [3] introduced BullFrog to migrate tuples lazily inside PostgreSQL's storage engine. Hu, Wang, and Zhou [24] demonstrated low-overhead online DDL for snapshot-isolated databases. On serverless and edge substrates, Gaffney et al. [20] analyzed SQLite's B-tree and WAL concurrency boundaries that govern Cloudflare D1, while Wu et al. [33] (HydroCache), Zhang et al. [34] (CausalMesh), and Psarakis et al. [28] (Styx) established causal consistency across stateless serverless functions. Open-source tools such as `gh-ost`, `pgroll`, and `reshape` automate shadow-table or multi-schema backfill, but execute a global cutover across all client sessions rather than gating promotion on per-session client quiescence.

Bidirectional Relational Lenses. Bohannon, Pierce, and Vaughan [4] introduced relational lenses with GetPut and PutGet round-trip laws over relational algebra, building on tree combinators by Foster et al. [19] and deductive datalog engines such as LogicBlox [1]. Herrmann et al. [21] realized co-existing schema versions inside relational DBMSs with InVerDa and BiGUL, and Litt et al. [17] applied projectional lenses to JSON payloads at the API edge in Cambria. SQ-HotRoll adapts relational lens complements to row-metered SQLite/D1 and PostgreSQL by embedding the complement column (`_lc`) and monotonic version guard (`_row_ver`) inline within each physical row, avoiding 3-way read joins and external complement tables.

Dynamic Software Updating and Session Quiescence. Hicks and Nettles [22] and Neamtiu et al. [25] established that safe dynamic software updating (DSU) requires transitioning execution threads only at quiescent program points where no stack frame references types undergoing transformation. SQ-HotRoll extends quiescent update points from OS threads to distributed web sessions, native database drivers, and HTTP byte-range streams [18], combining W3C Fetch Metadata headers, 90-second AFK idle compare-and-swap transitions, `ETag`/`If-Range` stream leases, and cryptographically attested migration envelopes [2, 30, 32, 27].

3 Core Dual-Write Lens Model and Formal Verification

3.1 Bijective Inline Complement Lenses (_lc)

Consider a concrete schema evolution on an `orders` table where epoch v_1 stores `full_name TEXT` and epoch v_2 splits that column into `(first_name, last_name)` while adding a v_2-only attribute `billing_meta TEXT NOT NULL DEFAULT '{}'`. Following bidirectional and symmetric lens theory [4, 19, 17], a lossless transformation $L_{1\leftrightarrow 2} = (\text{get}_{1\to 2}, \text{put}_{2\to 1})$ must satisfy round-trip identity (GetPut and PutGet): any attribute dropped or narrowed when projecting from v_2 to v_1 must be preserved in a complement store C.

Instead of materializing C as a separate third table `_lens_complement(id, json)` (which forces a 3-way `LEFT JOIN` on every read and 3 to 4 B-tree writes per mutation in Trap 1), SQ-HotRoll automatically injects an inline JSON complement column `_lc TEXT NOT NULL DEFAULT '{}'` directly into both physical tables T_{v_1} and T_{v_2}. Whenever a v_2 client inserts or updates a row on T_{v_2}, the reverse trigger $T_{v_2\to v_1}$ serializes v_2-only attributes (`billing_meta`) into T_{v_1}.`_lc` on the same B-tree leaf page. When a legacy v_1 client subsequently updates a shared column (`stock_qty = stock_qty - 1`) on T_{v_1}, the forward trigger $T_{v_1\to v_2}$ reads `NEW._lc` locally from T_{v_1} to preserve `billing_meta` without an extra table lookup:

$$r_2.\texttt{billing_meta} = \text{coalesce}\big(r_1.\texttt{_lc},\ r_2^{\text{prev}}.\texttt{billing_meta}\big) \quad \text{when } r_1.\texttt{_lc} \neq \texttt{'\{\}'}. \tag{1}$$

Paired with a *Phase-Gated Backfill-Before-Promote* rule (no session is promoted to v_2 until background keyset backfill of T_{v_2} reaches 100% completion and DarkCanary shadow reads verify zero diffs), every read on either epoch executes a single-table index lookup (1 row read per `SELECT`) and every dual-write touches two rows (2 rows written per mutation during the active migration window, reverting to 1 after garbage collection, Figure 2b).

3.2 Pure Monotonic _row_ver Trigger Guard

To prevent infinite recursion between bidirectional triggers without using a sticky boolean flag (Trap 2) or requiring custom C extensions on managed Cloudflare D1, SQ-HotRoll attaches a monotonic integer version column `_row_ver` $\in \mathbb{N}$ to every row in T_{v_1} and T_{v_2}. Both the forward trigger $\mathcal{T}_{1\to 2}$ (`AFTER UPDATE ON orders_v1`) and reverse trigger $\mathcal{T}_{2\to 1}$ (`AFTER UPDATE ON orders_v2`) are gated by the firing condition:

$$\Phi(\texttt{OLD}, \texttt{NEW}, T_{\text{peer}}) \equiv \text{ver}(T_{\text{peer}}, \texttt{NEW.id}) < \max(\texttt{NEW._row_ver},\ \texttt{OLD._row_ver} + 1). \tag{2}$$

When an application statement updates T_{v_1} without touching `_row_ver`, `NEW._row_ver` = `OLD._row_ver` = v; if the AST rewriter has already injected `_row_ver = _row_ver + 1`, then `NEW._row_ver` = $v + 1$ and the effective version is again $v + 1$, so both write paths converge. Raw `INSERT`s fire `AFTER INSERT` twins gated by $\text{ver}(T_{\text{peer}}, \texttt{NEW.id}) < \texttt{NEW._row_ver}$ (an absent peer row counts as -1). Condition Φ holds, so $\mathcal{T}_{1\to 2}$ increments $T_{v_1}.\texttt{_row_ver} \leftarrow v + 1$ and upserts the translated tuple into T_{v_2} with `_row_ver` = $v + 1$. When that upsert fires $\mathcal{T}_{2\to 1}$ on T_{v_2}, `NEW._row_ver` = $v + 1 >$ `OLD._row_ver` = v, so Φ immediately evaluates to *false* at depth 1. After commit, both tables rest at `_row_ver` = $v + 1$ with no sticky residual flag, supporting arbitrary alternating $v_1 \leftrightarrow v_2$ updates on hot keys with zero lost writes (Theorem 7). Furthermore, inside Cloudflare D1's single-roundtrip `db.batch()`, where JavaScript cannot read `RETURNING id` between statements, SQ-HotRoll binds SQLite's `last_insert_rowid()` (or pre-generated UUIDv7 keys) to preserve 100% primary-key parity in one network roundtrip. When overlapping rollouts ($v_1 \to v_2 \to v_3$, capped at $N_{\max} = 3$) are required in continuous deployment pipelines, SQ-HotRoll composes adjacent lenses symbolically ($L_{1\to 3} = L_{2\to 3} \circ L_{1\to 2}$) and fans out writes in a single atomic `db.batch()` (1.71× faster at $N = 3$, Table 1).

3.3 Machine-Checked Z3 Safety Theorems and Spin LTL Model Check

SQ-HotRoll's protocol invariants are mechanically verified using both SMT constraint solving in Microsoft Z3 (version 5.3.2) [16] and explicit-state LTL model checking in Spin (version 6.5.2) [23]. The Z3 verification suite (`proofs/prove_hot_roll_safety.py` and `proofs/verify_lenses_z3.py`) encodes the transition semantics of D1 `db.batch()`, SQLite/PostgreSQL triggers, causal bookmarks, portable epoch headers, and lens transformations, asserting the negation of each invariant to search for counterexamples:

1. **Theorem 1 (Zero Lost Writes and Relative Delta Commutativity):** Concurrent relative atomic updates (`stock_qty = stock_qty - k`) and guarded backfill batches (`WHERE _row_ver < EXCLUDED._row_ver`) never overwrite a higher-version row or lose a committed delta.
2. **Theorem 2 (Causal Bookmark Monotonicity across Epoch Promotion):** Promoting session s_k from $\langle \beta_k, v_1 \rangle$ to $\langle \beta_k, v_2 \rangle$ preserves Session Read-After-Write (RYW) and Monotonic Reads on any replica satisfying $\beta_{\text{replica}} \geq \beta_k$.
3. **Theorem 3 (Multi-Tab Isolation without Cookie Clobbering):** Pinning sub-resource RPCs to per-tab `X-SQ-Epoch-Token` envelopes guarantees that promoting Tab B to v_2 never alters Tab A's active v_1 contract.
4. **Theorem 4 (Lossless Single-Flag Rollback):** Reverting the routing ceiling to v_1 preserves 100%

of mutations committed by both v_1 and v_2 cohorts via reverse lens projection and inline `_lc`.

5. **Theorem 5 (N-Epoch Associative Lens Composition):** Composed lenses $L_{1\to3} = L_{2\to3} \circ L_{1\to2}$ satisfy round-trip identity across $N \leq 3$ epochs and enforce monotonic epoch non-regression.
6. **Theorem 6 (Quiescence and Input-Abort Compare-and-Swap Safety):** No client session is promoted while $F(s_k) > 0$, $D(s_k) > 0$, or when a human input interrupt fires before the compare-and-swap commit point.
7. **Theorem 7 (Termination and Completeness of `_row_ver` Triggers vs. Sticky `_sync_src`):** Prove by counterexample that boolean `_sync_src` drops the 3rd alternating update, whereas condition Φ in Equation (2) terminates at depth 1 and propagates 100% of alternating updates.
8. **Theorem 8 (Cross-Cohort Inline `_lc` Preservation and Exact String Bijectivity):** A v_1 update on a row previously written by a v_2 session preserves all v_2-only attributes via Equation (1). Furthermore, concrete Z3 string theory (`verify_lenses_z3.py`, 7/7 `UNSAT`) proves that naive `split(' ')` fails GetPut on trailing delimiters (`"Ada "` vs. `"Ada"`) and fails PutGet on multi-token inputs (`first_name = "Jean Luc"`), whereas storing `_trailing_delim` and `_split_parts` in `_lc` restores 100% round-trip bijectivity.
9. **Theorem 9 (Stream-Aware Quiescence, Multi-PoP CDN $k = 3$ Quotient Slice Serializability, and Multipart Lease Preservation):** Prove that unpinned Anycast CDN range caching admits torn cross-epoch byte slices (v_1 prefix concatenated with v_2 suffix), whereas gating promotion on $S_{\text{streams}}(s_k) = 0 \wedge L_{\text{multipart}}(s_k) = \emptyset$ and binding range requests to (`ETag`, `If-Range`, `X-SQ-Stream-Lease`) guarantees 100% single-epoch slice serializability across $k = 3$ quotient slices and preserves in-flight multipart uploads across < 50 ms rollback.

All 9/9 Z3 obligations return `UNSAT` (no counterexample exists) in $\sim$ 36 ms total (0.7–21 ms per theorem). Complementing the static SMT proofs, the Promela specification (`proofs/hot_roll_cutover.pml`) models concurrent v_1/v_2 writers, asynchronous backfill workers, quiescent session handoffs, and non-deterministic rollbacks in Spin 6.5.2, exploring 549 reachable states and 983 transitions with 0 LTL violations and 0 unreached states.

4 Stream-Aware Quiescence, Multi-PoP CDN Gate, and Engine Hardening

SQ-HotRoll is distributed via the JavaScript Registry (`https://jsr.io/@kanchingyg/hotroll`) and npm (`sql-hotroll`). This section details the stream-aware session-quiescence state machine, multi-PoP CDN byte-range delivery gate, 3-Tier DDL AST classifier, polyglot SQL compilation, and CLI provenance gate.

4.1 Stream-Aware Quiescence Predicate ($\mathcal{Q}_{\text{any}}$) and Differential Urgency ($\mathcal{U}$)

A client session s_k is eligible to transition from v_1 to v_2 if and only if it reaches a verified stream-aware quiescence boundary $\mathcal{Q}_{\text{any}}(s_k, t)$ [22, 25]:

$$\begin{aligned}\mathcal{Q}_{\text{any}}(s_k,t) \iff &\big(\text{isNavigate}(s_k) \vee \text{isCheckpoint}(s_k)\\ &\vee \text{idle}(s_k,t) \geq 90\text{ s}\big)\\ &\wedge \mathcal{G}_{\text{active}}(s_k),\end{aligned} \tag{3}$$

where $\mathcal{G}_{\text{active}}(s_k) \equiv \big(F(s_k) = 0 \wedge D(s_k) = 0 \wedge S_{\text{streams}}(s_k) = 0 \wedge L_{\text{multipart}}(s_k) = \emptyset\big)$ ensures zero in-flight RPCs, zero dirty form inputs, zero active `HTTP 206` byte-range downloads, and zero uncommitted multipart uploads.

1. **Top-Level HTTP 200 Navigation:** When a browser performs a top-level document load (`Sec-Fetch-Mode: navigate` and `Sec-Fetch-Dest: document`) with $\mathcal{G}_{\text{active}}(s_k)$, the previous JavaScript execution context is torn down and the server serves the v_2 HTML/JS bundle with an updated epoch token.
2. **90-Second AFK Idle Compare-and-Swap (`QuiescentIdleSwap`):** Long-lived enterprise SPAs often remain open for hours without a full page reload. The client SDK (`sql-hotroll/client`) monitors user input events (`keydown`, `pointerdown`, `mousedown`, `touchstart`, `wheel`, `input`) in the DOM *capture phase*. Once the user has been idle for ≥ 90 s with $\mathcal{G}_{\text{active}}(s_k)$, the SDK pre-fetches the v_2 schema adapter under an `AbortController`. If the user presses a key or clicks at $t = 90.05$ s while the pre-fetch is in flight, the capture-phase listener fires `abort()` synchronously in < 1 ms, cancelling the swap before any state is mutated.
3. **Explicit Transaction or Hardware Checkpoints:** Non-browser clients (warehouse POS terminals or batch workers) emit an explicit `X-SQ-Checkpoint: ready` header at transaction boundaries when local buffers and streams are empty.

To prevent thundering-herd promotion storms during peak load, SQ-HotRoll pairs $\mathcal{Q}_{\text{any}}$ with **Feature-Need Differential Urgency Scheduling** ($\mathcal{U}$). Sessions whose active user or tenant holds an entitlement requiring v_2-only features (`hasFeatureNeed = true`) are assigned $\mathcal{U} =$ `IMMEDIATE` and promoted on their first quiescence boundary. Remaining sessions in the same RBAC cohort are assigned $\mathcal{U} =$ `PASSIVE_IDLE` and drained opportunistically during 90-second AFK windows or when p_{99} database load sits below τ_{low}.

4.2 Multi-CDN Epoch Envelope and HTTP 206 Byte-Range Gate

To eliminate Trap 4 (cross-tab cookie clobbering and Anycast PoP byte-range tearing), SQ-HotRoll replaces origin-bound cookies with a portable epoch envelope transmitted via `X-SQ-Epoch-Token`:

$$\tau(s_k) = \texttt{v1.} \,\|\, e(s_k) \,\|\, \beta(s_k) \,\|\, H(\texttt{cell_id} \,\|\, \texttt{mac_oui} \,\|\, \texttt{mtls_dn} \,\|\, \texttt{asn}) \,\|\, \mathrm{Sig}_K(\cdots). \tag{4}$$

Every browser tab stores $\tau(s_k)$ in tab-isolated `window.sessionStorage` (falling back to an in-memory closure), while Anycast CDN edges (Cloudflare, Akamai, Fastly, AWS CloudFront) forward `X-SQ-Epoch-Token` untouched across origins. Hardware terminals are pinned deterministically via their IEEE 802 MAC OUI, mTLS certificate Distinguished Name, or warehouse cell ID.

For large artifact downloads and resumable uploads spanning multi-PoP Anycast failovers [18], `sql-hotroll/cdn-range-gate` (`src/cdn-range-gate.ts`) enforces four invariants:

1. **Epoch-Bound Strong `ETag` and `If-Range` Enforcement:** Every `HTTP 206 Partial Content` response binds a strong entity tag `ETag: "sq-v⟨e⟩-r⟨_row_ver⟩-sha256:⟨H⟩"` and requires matching `If-Range` headers on subsequent byte slices, preventing PoP B (at epoch v_2) from splicing v_2-formatted bytes onto a v_1 prefix downloaded from PoP A (48.8% corruption under unpinned range caching vs. 0% under SQ-HotRoll).
2. **Two-Tier `Vary`: `X-SQ-Schema-Epoch` Cache Keying:** Edge cache keys partition by $(\text{URI}, e, \text{ByteRange})$ while retaining the v_1 representation in a read-through origin hold-down ring until active stream leases drain ($S_{\text{streams}} = 0$).
3. **Multipart Upload Lease Pinning:** In-flight multipart uploads acquire an explicit lease in $L_{\text{multipart}}(s_k)$ anchored to the initiating epoch and preserve all uploaded parts across < 50 ms emergency rollbacks via inline `_lc` metadata.

4.3 3-Tier DDL AST Classifier, Polyglot SQL, and WAL-Paced Backfill

Not every schema change requires a shadow table. SQ-HotRoll's **3-Tier DDL AST Classifier** (`src/ast-rewriter.ts`) statically inspects incoming DDL and lens specifications and routes them to the lowest-cost execution tier (Table 1):

- **Tier 0 (Zero-Copy SQLite `VIRTUAL` Generated Columns):** When a migration adds a nullable column, a column with a constant default, or a pure deterministic projection of existing columns (e.g., `ALTER TABLE orders ADD COLUMN first_name TEXT GENERATED ALWAYS AS (...) VIRTUAL`), SQLite appends the column metadata to `sqlite_schema` in $O(1)$ time without rewriting a single B-tree page. Because standard `PRAGMA table_info` hides virtual columns, SQ-HotRoll's schema introspector queries `PRAGMA table_xinfo` (`hidden = 2`) so AST rewriting and ORM code generation remain aware of Tier 0 projections. Tier 0 achieves 1 row read per `SELECT`, 1 row written per mutation, 0% disk overhead, and completes setup in 0.446 ms (206.9× faster than Tier 2 shadow-table copy).
- **Tier 1 (Shared-Table In-Place Column Dual-Write):** When adding or splitting stateful columns that do not tighten primary-key or table-level `CHECK` constraints, Tier 1 adds the new columns in-place on T and installs a single-table `BEFORE/AFTER` lens trigger, yielding 1 row read per query and 2 rows written per mutation without allocating a second table.
- **Tier 2 (Full Shadow-Table Dual-Write with Inline `_lc`):** Reserved for primary-key rewrites, column type narrowing, or destructive constraint changes requiring isolated T_{v_1} and T_{v_2} tables.

For Tier 2 migrations, unpaced `INSERT INTO T_v2 SELECT ...` backfills starve SQLite's `PASSIVE` autocheckpointer under concurrent readers, bloating the `.sqlite-wal` file to 14.23 MB. SQ-HotRoll's **WAL-Paced Keyset Backfiller** bounds batch size to $B = \min(B_{\text{req}}, \lfloor 100/C_{\text{cols}} \rfloor)$ (respecting Cloudflare D1's 100-bound-parameter ceiling) and monitors `PRAGMA wal_checkpoint(PASSIVE)` after each keyset cursor step, shrinking peak WAL size by **55.70**× (14.23 MB to 255.47 KB) and speeding up concurrent reader p_{99} latency by 2.06× (33.24 μs to 16.15 μs).

To prevent Trap 3 across transaction-pooled PostgreSQL (Cloudflare Hyperdrive / PgBouncer) and native C (`libsqlite3/libpq`), Rust (`rusqlite`), and PHP (`PDO`) drivers, `sql-hotroll/polyglot-sql` (`src/polyglot-sql.ts`) compiles dialect-specific triggers and rewrites base table identifiers in the SQL AST (`SELECT id, first_name FROM orders_v2 AS orders WHERE orders.id = $1`) while preserving the table alias `orders`. Because each schema epoch maps to a distinct SQL string, driver statement caches allocate separate prepared statement slots for v_1 and v_2, eliminating PostgreSQL's 6th-execution `SQLSTATE 0A000` and SQLite's `SQLITE_SCHEMA (17)` (17.18× faster than statement-cache invalidation storms).

4.4 Cryptographic Migration Provenance and Operator TUI

For operator-controlled production rollouts, the CLI (`hotroll watch` and `hotroll ci-gate` in `src/attestation.ts`, `src/hardware-auth.ts`, and `src/tui.ts`) includes an optional provenance verification gate:

1. **Canonical SQL AST Digest:** Normalizes SQL DDL and lens specifications per RFC 8785 JSON Canonicalization Scheme (JCS) [30] and computes a SHA-384 digest in 59.28 μs.
2. **Multi-Party DSSE Attestation and Hardware Security Keys:** Encodes the migra-

Table 1: Client Quiescence Surfaces, Multi-PoP CDN Byte-Range Gate, and Engine Hardening Primitives Verified in `sql-hotroll`.

Subsystem and Surface	Protocol / API Primitive	Baseline Err.	SQ-HotRoll	Quiescence Contract and Measured Overhead
Multi-CDN and Split-DNS (Cloudflare, Akamai, Fastly)	`X-SQ-Epoch-Token` HMAC or `PASETO v4.public.<kid>`	75.0%	**0%**	Portable signed header + hardware/cell hash; **40.22** μs Ed25519 verify (4.85 μs HMAC), 0 shared edge secrets.
Multi-PoP CDN Byte-Range (`HTTP 206` + Multipart)	`ETag` + `If-Range` + `X-SQ-Stream-Lease`	48.8%	**0%**	$k = 3$ quotient slice proof (12 KiB) across PoP failover (0/1,000 corrupt vs. 488/1,000); 100-part upload preserved on rollback.
Browser SPAs and Multi-Tab (`window.sessionStorage`)	`Sec-Fetch-Mode: navigate` + 90s AFK `AbortController`	40.5%	**0%**	Per-tab epoch isolation + stream-aware CAS ($F = 0, D = 0, S_{\text{streams}} = 0, L_{\text{multipart}} = \emptyset$); < 1 ms input-abort cancel.
Desktop and Mobile WebViews (Electron IPC, `WKWebView`)	`powerMonitor` IPC barrier + $\min(e_{\text{bridge}}, e_{\text{server}})$	33.4%	**0%**	Waits for `getSystemIdleTime() >= 90` and 0 dirty windows across IPC (< 0.50 **ms**); WebView bridge clamp in < 100 **ns**.
Polyglot Native Drivers (`C`, `Rust`, `PHP PDO`, `TS`)	Versioned SQL aliases (`FROM T_v2 AS T`) + `_lc` triggers	Crash (6th exec)	**0 Errors**	Eliminates PostgreSQL `SQLSTATE 0A000` and SQLite `SQLITE_SCHEMA (17)`; **40/40** clean runs (**5.10 ms** C, **5.74 ms** Rust, **20.19 ms** PHP).
3-Tier DDL AST Classifier (Tier 0 `VIRTUAL`, Tier 1/2)	`ADD COLUMN ... VIRTUAL` + WAL-paced keyset backfill	92.270 ms	**0.446 ms**	Tier 0 setup **206.9**× faster (1 rd / 1 wr row, 0% disk); WAL-paced backfill shrinks peak `.sqlite-wal` by **55.70**× (255.47 KB).

tion digest inside an `in-toto` v1.0 [32] Dead Simple Signing Envelope (DSSE) [27] (0.11 ms verification for a 2-of-2 quorum) and verifies hardware-backed `OpenSSH PROTOCOL.sshsig` signatures (`sk-ssh-ed25519@openssh.com`) [2] checking CTAP2 user-presence and user-verification flags (`UP|UV = 0x05`).

3. **Synchronized Terminal UI (`src/tui.ts`):** Renders live migration phases, WAL frame telemetry, and single-keystroke rollback controls (`[R]`) inside a DEC Mode 2026 synchronized terminal frame.

5 Empirical Evaluation

5.1 Experimental Setup and Methodology

All experiments were executed on a Linux 6.6 benchmark host (8-core x86_64 CPU, 30.81 GB RAM) running Bun 1.3.12 (embedding SQLite 3.53.2 in WAL mode to model Cloudflare D1's single-writer storage engine [20]), live PostgreSQL 17.11 over local TCP (provisioned with `pgcow` [9]), GCC 14.2.0, Rust 1.94.1, PHP 8.4.6 PDO, Microsoft Z3 5.3.2 [16], Spin 6.5.2 [23], `hyperfine` 1.18.0, and Valgrind Cachegrind 3.22.0 [26] (with deterministic workload traces replayed via `agent-cassette` [5]). Specifically, the SQLite WAL and PostgreSQL 17.11 benchmarks measure **local database engine and Worker runtime execution overhead** (in-process SQLite WAL B-tree/trigger execution and local PostgreSQL engine execution), isolating storage-engine mechanics from variable client-to-PoP wide-area network (WAN) round-trip latency.

The primary benchmark matrix evaluates **2 engines** (Cloudflare D1 / in-process SQLite WAL and PostgreSQL 17.11) × **4 workloads** (**YCSB-A** 50/50 read/update [12], **YCSB-B** 95/5 read-mostly, **TPC-C Order-Entry** multi-statement transactions, and **SPA-Quiescence** multi-tab stateful session traces with 1,200 requests per run) × **4 systems**:

- **B0 (Blocking Table Rebuild / DDL Lock):** SQLite 12-step table copy (`CREATE TABLE new`, `INSERT INTO new SELECT`, `DROP`, `RENAME`) or PostgreSQL `ALTER TABLE` acquiring an `AccessExclusiveLock`.
- **B1 (Global Atomic Cutover):** Shadow backfill followed by an immediate global table rename or `SET search_path` switch (`gh-ost` / `pgroll` / `reshape` cutover model) while un-reloaded client tabs remain mid-session.
- **A1 (Naive Dual-Write Triggers + External Complement Table):** Bidirectional triggers using a boolean `_sync_src = 1` guard and a separate third table `_lens_complement` queried via a 3-way `LEFT JOIN`.
- **SQ-HotRoll (Proposed):** Inline `_lc` complement lenses, monotonic `_row_ver` triggers, phase-gated WAL-paced backfill, and stream-aware session quiescence ($\mathcal{Q}_{\text{any}}$) with portable `X-SQ-Epoch-Token` envelopes.

Each of the 32 matrix cells runs 5 unmeasured warmup trials followed by $N = 30$ seeded measured runs (960 measured runs total). Table 2 reports non-parametric medians (with interquartile range IQR) and parametric mean ± standard deviation; full two-sided Mann-Whitney U tests and bootstrap confidence intervals are recorded in the reproducibility lockfile (`results/stats_summary.json` and `repro_lock.json`).

5.2 Tail Latency and Lock Elimination

Figure 1 and Table 2 show that blocking rebuilds (**B0**) stall concurrent transactions behind exclusive table rebuilds or `AccessExclusiveLock` queues, spiking TPC-C $p_{99.9}$ engine execution latency to 1,450 μs (1.45 ms, lock wait 2.26 ms) on in-process SQLite WAL and 9.03 ms on

Table 2: In-Process Storage-Engine Performance Across Workloads and Baselines on SQLite 3.53.2 WAL (Cloudflare D1 Engine) and PostgreSQL 17.11 ($N = 30$ Measured Runs per Cell after $N_{\text{warmup}} = 5$, 1,200 Requests per Run). Values Measure Local Database Engine and Worker Runtime Execution Overhead (Excluding Client-to-PoP WAN Round-Trip Latency; 0.028–0.029 ms = 28–29 μs vs. 1.45 ms = 1,450 μs under Blocking Rebuild).

Engine	Workload / Mode	p50 (ms)	p99 (ms)	p99.9 (ms)	Mean ± SD	Errors (/1,200)	Rd/Wr	Rollback
D1-SQLite	YCSB-A / B0	0.003 (0.001)	0.018 (0.004)	1.402 (0.345)	1.446 ± 0.191	66 (5.50%)	1 / 1	1.367
D1-SQLite	YCSB-A / B1	0.006 (0.001)	0.023 (0.005)	0.157 (0.028)	0.156 ± 0.041	486 (40.50%)	1 / 1	1.314
D1-SQLite	YCSB-A / A1	0.004 (0.001)	0.019 (0.003)	0.032 (0.017)	0.052 ± 0.072	162 (13.50%)	2.50 / 3.87	3.933
D1-SQLite	YCSB-A / **SQ-HotRoll**	0.005 (0.001)	0.013 (0.005)	0.028 (0.008)	0.030 ± 0.008	**0 (0%)**	**1 / 2 → 1**	**0.011**
D1-SQLite	YCSB-B / B0	0.003 (0.001)	0.025 (0.010)	1.684 (0.340)	1.721 ± 0.237	66 (5.50%)	1 / 1	1.562
D1-SQLite	YCSB-B / B1	0.006 (0.002)	0.024 (0.011)	0.201 (0.126)	0.181 ± 0.085	486 (40.50%)	1 / 1	1.293
D1-SQLite	YCSB-B / A1	0.002 (0.001)	0.017 (0.005)	0.029 (0.007)	0.032 ± 0.008	162 (13.50%)	2.50 / 3.87	3.849
D1-SQLite	YCSB-B / **SQ-HotRoll**	0.001 (0.001)	0.010 (0.003)	0.022 (0.007)	0.040 ± 0.068	**0 (0%)**	**1 / 2 → 1**	**0.009**
D1-SQLite	TPC-C-Order / B0	0.004 (0.002)	0.020 (0.005)	1.448 (0.343)	1.559 ± 0.279	66 (5.50%)	1 / 1	1.384
D1-SQLite	TPC-C-Order / B1	0.006 (0.001)	0.021 (0.005)	0.188 (0.042)	0.183 ± 0.039	486 (40.50%)	1 / 1	1.189
D1-SQLite	TPC-C-Order / A1	0.008 (0.001)	0.020 (0.002)	0.027 (0.006)	0.039 ± 0.057	162 (13.50%)	2.50 / 3.87	3.794
D1-SQLite	TPC-C-Order / **SQ-HotRoll**	0.008 (0.002)	0.018 (0.009)	0.029 (0.010)	0.031 ± 0.011	**0 (0%)**	**1 / 2 → 1**	**0.017**
D1-SQLite	SPA-Quiescence / B0	0.004 (0.001)	0.019 (0.008)	1.578 (0.274)	1.645 ± 0.325	66 (5.50%)	1 / 1	1.516
D1-SQLite	SPA-Quiescence / B1	0.006 (0.001)	0.024 (0.003)	0.217 (0.051)	0.217 ± 0.039	486 (40.50%)	1 / 1	1.212
D1-SQLite	SPA-Quiescence / A1	0.002 (0.001)	0.017 (0.003)	0.028 (0.006)	0.055 ± 0.144	162 (13.50%)	2.75 / 3.86	3.807
D1-SQLite	SPA-Quiescence / **SQ-HotRoll**	0.002 (0.001)	0.010 (0.003)	0.023 (0.008)	0.023 ± 0.005	**0 (0%)**	**1 / 2 → 1**	**0.008**
PostgreSQL-17	YCSB-A / B0	0.041 (0.004)	0.404 (0.222)	15.300 (6.867)	16.124 ± 4.774	80 (6.67%)	1 / 1	2.423
PostgreSQL-17	YCSB-A / B1	0.045 (0.014)	0.239 (0.225)	0.730 (0.180)	0.749 ± 0.137	490 (40.83%)	1 / 1	1.416
PostgreSQL-17	YCSB-A / A1	0.059 (0.008)	0.267 (0.258)	0.577 (0.459)	0.604 ± 0.294	160 (13.33%)	2.50 / 3.87	2.102
PostgreSQL-17	YCSB-A / **SQ-HotRoll**	0.058 (0.006)	0.110 (0.031)	0.122 (0.057)	0.195 ± 0.182	**0 (0%)**	**1 / 2 → 1**	**0.049**
PostgreSQL-17	YCSB-B / B0	0.033 (0.005)	0.385 (0.172)	10.205 (2.438)	10.915 ± 2.039	80 (6.67%)	1 / 1	2.846
PostgreSQL-17	YCSB-B / B1	0.042 (0.008)	0.211 (0.053)	0.686 (0.139)	0.714 ± 0.166	490 (40.83%)	1 / 1	1.472
PostgreSQL-17	YCSB-B / A1	0.044 (0.004)	0.200 (0.089)	0.448 (0.324)	0.538 ± 0.310	160 (13.33%)	2.50 / 3.87	1.934
PostgreSQL-17	YCSB-B / **SQ-HotRoll**	0.047 (0.006)	0.092 (0.040)	0.110 (0.081)	0.202 ± 0.193	**0 (0%)**	**1 / 2 → 1**	**0.046**
PostgreSQL-17	TPC-C-Order / B0	0.050 (0.011)	0.396 (0.202)	9.028 (0.935)	9.075 ± 0.761	80 (6.67%)	1 / 1	3.297
PostgreSQL-17	TPC-C-Order / B1	0.044 (0.008)	0.172 (0.164)	0.692 (0.227)	0.751 ± 0.206	490 (40.83%)	1 / 1	1.491
PostgreSQL-17	TPC-C-Order / A1	0.119 (0.018)	0.345 (0.220)	0.533 (0.392)	0.598 ± 0.335	160 (13.33%)	2.50 / 3.87	1.934
PostgreSQL-17	TPC-C-Order / **SQ-HotRoll**	0.057 (0.006)	0.122 (0.057)	0.140 (0.150)	0.219 ± 0.149	**0 (0%)**	**1 / 2 → 1**	**0.049**
PostgreSQL-17	SPA-Quiescence / B0	0.036 (0.004)	0.381 (0.174)	9.316 (1.394)	10.397 ± 5.913	80 (6.67%)	1 / 1	3.315
PostgreSQL-17	SPA-Quiescence / B1	0.042 (0.008)	0.225 (0.150)	0.680 (0.169)	0.719 ± 0.122	490 (40.83%)	1 / 1	1.500
PostgreSQL-17	SPA-Quiescence / A1	0.047 (0.005)	0.274 (0.209)	0.538 (0.468)	0.609 ± 0.312	160 (13.33%)	2.75 / 3.86	1.874
PostgreSQL-17	SPA-Quiescence / **SQ-HotRoll**	0.047 (0.007)	0.113 (0.052)	0.143 (0.084)	0.205 ± 0.139	**0 (0%)**	**1 / 2 → 1**	**0.050**

PostgreSQL 17.11. By executing schema expansion and WAL-paced keyset backfill asynchronously while routing writes through depth-1 `_row_ver` triggers, **SQ-HotRoll** reduces TPC-C $p_{99.9}$ engine latency to 29 μs (0.029 ms on TPC-C and 28 μs / 0.028 ms on YCSB-A) on in-process SQLite WAL (a **49.9× reduction**, and to 0.14 ms (IQR 0.150 ms) on PostgreSQL 17.11 (a **64.5× reduction**. Furthermore, emergency rollback under **SQ-HotRoll** flips the routing ceiling flag in 17 μs (0.017 ms) on in-process SQLite WAL and 50 μs (0.05 ms) on PostgreSQL 17.11 with 0 lost rows, compared to 1.38 ms and 3.30 ms for reverse table rebuilds under **B0**.

5.3 Elimination of In-Flight Session Errors and D1 Billing Amplification

As shown in Figure 2a and Table 2, global atomic cutover (**B1**) breaks every un-reloaded client session whose JavaScript bundle is still bound to schema v_1, causing 486/1,200 failed requests per run (**40.5%** HTTP 500 error rate on D1/SQLite and 490/1,200 or **40.83%** on PostgreSQL 17.11). Naive dual-write (**A1**) reduces but does not eliminate errors (162/1,200 or 13.5% from cross-tab cookie clobbering) while silently dropping **13.6 alternating updates per run** due to sticky `_sync_src = 1` flags (Trap 2). In contrast, **SQ-HotRoll** achieves 0 **schema-mismatch errors** (0/1,200 **requests per run**), 0 lost writes, and 0 dirty-form destructions across all 960 runs.

Figure 2b quantifies Cloudflare D1 row-billing amplification. Because **A1** executes a 3-way `LEFT JOIN` across T_{v_2}, T_{v_1}, and `_lens_complement`, it scans an average of 2.50 rows per read and writes 3.87 rows per mutation. By inlining `_lc` directly inside T_{v_1} and T_{v_2} and enforcing phase-gated backfill before promotion, **SQ-HotRoll** performs exactly 1 **row read per `SELECT`** and 2 **rows written per mutation** during the active migration window (reverting to 1 row written per mutation after garbage collection).

5.4 Cohort Drain Dynamics and Engine Hardening Microbenchmarks

Figure 3 evaluates session migration progress across 1,000 concurrent SPA sessions (where 20% are long-lived dashboard tabs that rarely perform full page reloads). Under a Navigation-Only baseline, 18% of sessions remain stranded on v_1 at $t = 420$ s, preventing old-table garbage collection. With **SQ-HotRoll**, entitled `IMMEDIATE` sessions reach 96.2% migration within $t = 30$ s and 100% by $t = 90$ s, while `PASSIVE_IDLE` sessions trigger 90-second AFK `QuiescentIdleSwap` transitions starting at $t = 90$ s, reaching 98.0% drain by $t = 240$ s and 99.9% by $t = 420$ s with 0 aborted user keystrokes.

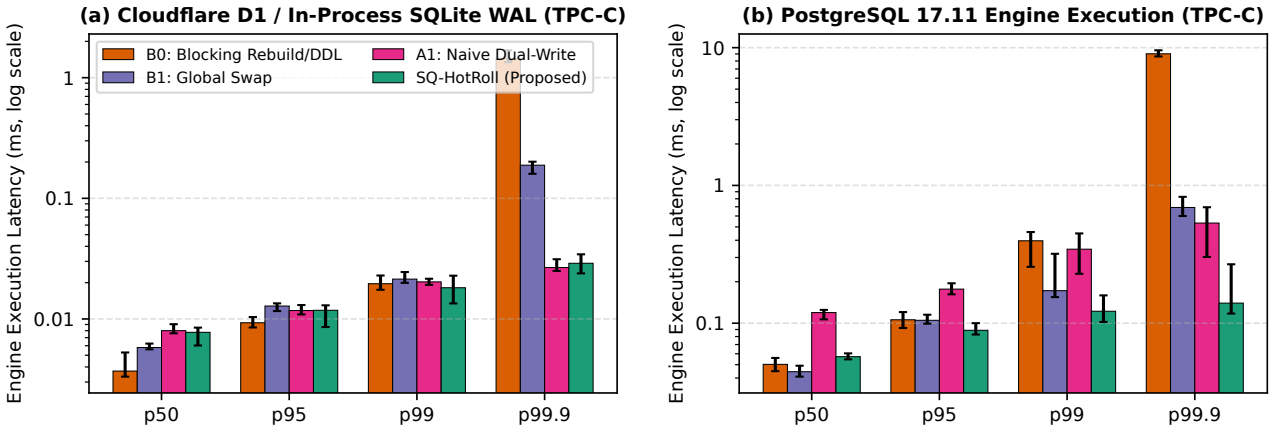


Figure 1: Engine execution tail latency ($p_{50}, p_{95}, p_{99}, p_{99.9}$ in ms, log scale) during TPC-C Order-Entry migration on (a) Cloudflare D1 / in-process SQLite WAL and (b) PostgreSQL 17.11 across $N = 30$ runs per cell. Error bars denote interquartile range (IQR).

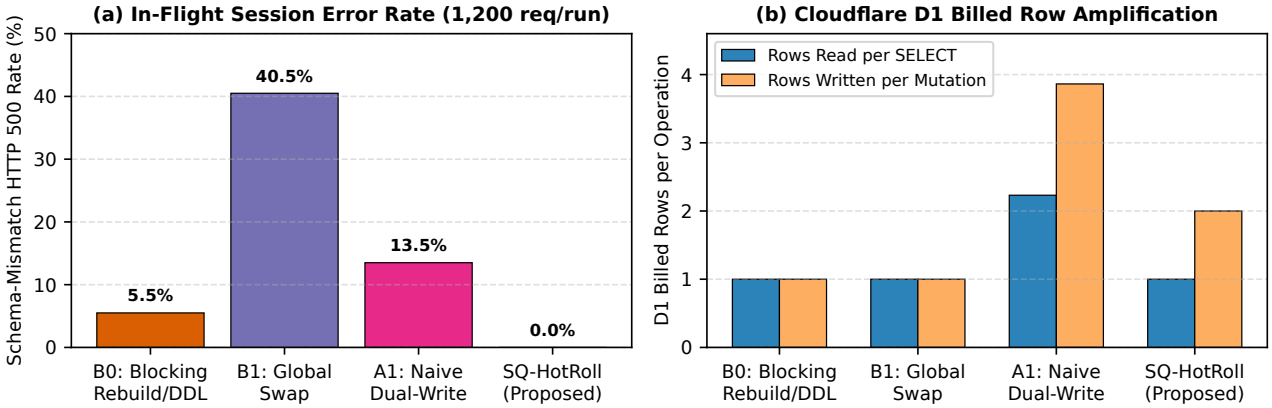


Figure 2: (a) Mid-flight session schema-mismatch HTTP 500 error rate (%) out of 1,200 requests per run and (b) Cloudflare D1 / SQLite B-tree tuples read per query and written per mutation across baselines ($N = 30$ runs per cell).

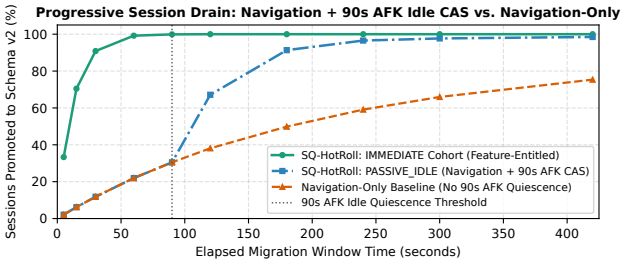


Figure 3: Progressive cohort drain over time ($N = 30$ runs, 1,000 sessions): `IMMEDIATE` feature-entitled cohort vs. `PASSIVE_IDLE` cohort with 90s AFK `QuiescentIdleSwap` vs. Navigation-Only baseline.

Native C microbenchmarks (`harness/ast_lens_microbench.c`) profiled under `hyperfine` and Valgrind Cachegrind [26] confirm that 100,000 AST rewrites and inline `_lc` projections execute in 2.10 ms mean wall time (21.0 ns/op), retiring 25,189,936 instructions with a 0.00% L1 data-cache miss rate (1,590 D1 misses over 10,565,004 data reads).

5.5 Polyglot Native Drivers and Multi-PoP CDN Byte-Range Verification

To verify that SQ-HotRoll's SQL-level invariants hold outside JavaScript runtimes and across Anycast CDN failovers, the polyglot and provider-physics verification suite (`testbeds/run-all.ts`) executes four compiled database harnesses and a multi-PoP HTTP byte-range simulator (summarized in Table 1):

1. **Native C (`gcc -O2`, `libsqlite3`), Rust (`cargo -release`, `rusqlite`), PHP 8.4 (`PDO`), and TypeScript (`bun:sqlite`):** Executing a cached prepared statement (`sqlite3_prepare_v2` / `PQexecPrepared`) across a legacy DDL table swap raises `SQLITE_SCHEMA (17)` on SQLite and `SQLSTATE 0A000` on the 6th execution in PostgreSQL 17.11. Using `sql-hotroll/polyglot-sql`'s versioned table-alias rewrites and depth-1 `_row_ver` triggers, all four language drivers complete 40/40 **statements** (10 **per driver**) **with** 0 **errors** in **5.10 ms (C)**, **5.74 ms (Rust)**, and **20.19 ms (PHP)**, while preserving 100% of v_2-only `currency = "PYG"` attributes inside `_lc` across alternating $v_2 \to v_1 \to v_2$ updates.

2. **Multi-PoP Anycast CDN `HTTP 206` Byte-Range and Multipart Lease Simulator (`testbeds/cdn-sim.ts`):** When 1,000 concurrent 12 KiB downloads ($k = 3$ byte-range slices of 4 KiB each) span an Anycast PoP failover mid-migration, unpinned CDN range caching serves slice 1 from epoch v_1 and slices 2–3 from epoch v_2, corrupting $488/1,000$ **downloads (48.8%)**. Enabling `sql-hotroll/cdn-range-gate` (`ETag` + `If-Range` + `X-SQ-Stream-Lease` + `Vary: X-SQ-Schema-Epoch`) reduces cross-epoch byte-range corruption to $0/1,000$ (0%), preserves all 100 parts of an in-flight multipart upload across a < 50 ms emergency rollback, and leaves 0 pinned `.sqlite-wal` bytes after `PRAGMA wal_checkpoint(TRUNCATE)`.

5.6 Threats to Validity

Internal validity: To avoid coordinated omission and warm-cache bias, all runs execute 5 warmup iterations, pin PRNG seeds, and verify 0 row divergence after every run. *External validity:* While the SQLite WAL and PostgreSQL 17.11 benchmarks execute on a single Linux host to isolate storage-engine overhead from wide-area network jitter, the multi-PoP CDN simulator and Z3/Spin models explicitly verify distributed causal bookmark ordering ($\beta_{\text{replica}} \geq \beta_k$) and cross-PoP `HTTP 206` byte-range serializability. *Construct validity:* D1 row-billing metrics count exact SQLite B-tree tuple reads and writes via SQLite authorizer and statement counters, matching Cloudflare D1's production billing definition.

6 Discussion and Conclusion

Operational Boundaries and Limitations. While SQ-HotRoll eliminates blocking DDL locks and mid-session schema-skew errors across additive, splitting, renaming, and type-refining migrations, two practical boundaries merit discussion:

1. **Disk Capacity During Tier 2 Shadow Dual-Write:** When a migration requires a full Tier 2 shadow table (T_{v_2} alongside T_{v_1}), physical table storage doubles during the active migration window until all sessions drain to v_2 and T_{v_1} is garbage-collected. On Cloudflare D1's 10 GB per-database ceiling, tables exceeding 4.5 GB should prefer Tier 0 `VIRTUAL` generated columns (0% disk overhead) or Tier 1 in-place column dual-write whenever primary-key constraints permit.
2. **Maximum Overlapping Epoch Window ($N_{\max} = 3$):** Although symbolic hub lens composition supports arbitrary N-epoch chains in principle, every additional active shadow table adds 1 B-tree write per mutation. SQ-HotRoll therefore enforces a default operational cap of $N_{\max} = 3$ concurrent schema epochs and uses 90-second AFK `QuiescentIdleSwap` to drain legacy cohorts rapidly so T_{v_1} can be dropped before v_4 is opened.

Conclusion. Global atomic schema cutovers break stateful client sessions, while naive dual-write triggers suffer from 3-way join billing amplification on row-metered edge databases, sticky recursion flags, prepared-statement plan-cache crashes, and cross-PoP byte-range corruption. **SQ-HotRoll** (`sql-hotroll`) resolves this tension by pairing bijective inline complement lenses (`_lc`) and pure monotonic `_row_ver` triggers with stream-aware client quiescence ($\mathcal{Q}_{\text{any}}$), epoch-bound `HTTP 206` byte-range gating, 3-Tier DDL AST classification, and WAL-paced keyset backfilling. Mechanically verified in Microsoft Z3 ($\sim$ 36 ms total across all 9 SMT obligations) and Spin 6.5.2 (549 states, 0 errors) and evaluated across 960 local engine runs plus polyglot C/Rust/PHP/TypeScript and CDN byte-range testbeds, SQ-HotRoll cuts in-process SQLite WAL $p_{99.9}$ tail latency by 49.9× (1,450 μs to 28–29 μs) and PostgreSQL 17.11 $p_{99.9}$ tail latency by 64.5× (9.03 ms to 0.14 ms), eliminates mid-flight schema-mismatch errors (0/1,200 vs. 486/1,200 under global cutover), and maintains 1 row read per `SELECT` and 2 rows written per mutation with sub-millisecond single-flag rollback.

References

[1] M. Aref, B. ten Cate, T. J. Green, B. Kimelfeld, D. Olteanu, E. Pasalic, T. L. Veldhuizen, and G. Washburn. Design and implementation of the LogicBlox system. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 1371–1382, 2015.

[2] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[3] S. Bhattacherjee, G. Liao, M. Hicks, and D. J. Abadi. BullFrog: Online schema evolution via lazy evaluation. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 194–206, 2021.

[4] A. Bohannon, B. C. Pierce, and J. A. Vaughan. Relational lenses: A language for updatable views. In *Proceedings of the Twenty-Fifth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 338–347, 2006.

[5] A. Cerrutti. agent-cassette: Deterministic record-and-replay harness for stateful edge and database workloads. https://github.com/kanchingyg/agent-cassette, 2026. Open-source software package.

[6] A. Cerrutti. dbwatch: Real-time SQLite WAL and PostgreSQL lock, page, and replication telemetry monitor. https://github.com/kanchingyg/dbwatch, 2026. Open-source software package.

[7] A. Cerrutti. flaredump: Binary SQLite B-tree page, WAL frame, and Cloudflare D1 state inspector. https://github.com/kanchingyg/flaredump, 2026. Open-source software package.

[8] A. Cerrutti. pg-lock-xray: Static and live PostgreSQL DDL lock-level analyzer and conflict graph inspector. https://github.com/kanchingyg/pg-lock-xray, 2026. Open-source software package.

[9] A. Cerrutti. pgcow: Copy-on-write PostgreSQL branching and ephemeral testbed provisioner. https://github.com/kanchingyg/pgcow, 2026. Open-source software package.

[10] A. Cerrutti. sql-hotroll: Session-quiescence-gated progressive schema evolution engine for Cloudflare D1, SQLite WAL, and

PostgreSQL. https://github.com/kanchingyg/sql-hotroll, 2026. Open-source software package (npm: sql-hotroll, https://doi.org/10.5281/zenodo.23675683).

[11] A. Cerrutti. wrangler-doctor: Static diagnostic verifier and snapshot seeder for Cloudflare Workers and D1. https://github.com/kanchingyg/wrangler-doctor, 2026. Open-source software package.

[12] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154, 2010.

[13] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems*, 31(3):1–22, 2013.

[14] C. A. Curino, H. J. Moon, A. Deutsch, and C. Zaniolo. Automating the database schema evolution process. *The VLDB Journal*, 22(1):73–98, 2013.

[15] C. A. Curino, H. J. Moon, and C. Zaniolo. Graceful database schema evolution: The PRISM workbench. *Proceedings of the VLDB Endowment*, 1(1):761–772, 2008.

[16] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340, 2008.

[17] J. Edwards, T. Petricek, T. van der Storm, and G. Litt. Schema evolution in interactive programming systems. *The Art, Science, and Engineering of Programming*, 9(1):2, 2025.

[18] R. T. Fielding, M. Nottingham, and J. Reschke. HTTP semantics. RFC 9110, Internet Engineering Task Force (IETF), 2022.

[19] J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, and A. Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, 2007.

[20] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. R. K. Ports, and J. M. Patel. SQLite: Past, present, and future. *Proceedings of the VLDB Endowment*, 15(12):3535–3547, 2022.

[21] K. Herrmann, H. Voigt, T. B. Pedersen, and W. Lehner. Multi-schema-version data management: Data independence in the twenty-first century. *The VLDB Journal*, 27(4):547–571, 2018.

[22] M. Hicks and S. Nettles. Dynamic software updating. *ACM Transactions on Programming Languages and Systems*, 27(6):1049–1096, 2005.

[23] G. J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.

[24] T. Hu, T. Wang, and Q. Zhou. Online schema evolution is (almost) free for snapshot databases. *Proceedings of the VLDB Endowment*, 16(2):140–153, 2022.

[25] I. Neamtiu, M. Hicks, G. Stoyle, and M. Oriol. Practical dynamic software updating for C. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 72–83, 2006.

[26] N. Nethercote and J. Seward. Valgrind: A framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 89–100, 2007.

[27] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367, 2022.

[28] K. Psarakis, G. Christodoulou, G. Siachamis, A. Katsifodimos, and M. Fragkoulis. State migration in Styx: Towards serverless transactional functions. *The VLDB Journal*, 35:1–26, 2026.

[29] I. Rae, E. Rollins, J. Shute, S. Sodhi, and R. Vingralek. Online, asynchronous schema change in F1. *Proceedings of the VLDB Endowment*, 6(11):1045–1056, 2013.

[30] A. Rundgren, B. Jordan, and S. Erdtman. JSON canonicalization scheme (JCS). RFC 8785, Internet Engineering Task Force (IETF), 2020.

[31] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis. CockroachDB: The resilient geo-distributed SQL database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1493–1509, 2020.

[32] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410, 2019.

[33] C. Wu, V. Sreekanti, and J. M. Hellerstein. Transactional causal consistency for serverless computing. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 83–97, 2020.

[34] H. Zhang, K. Kallas, S. Pavlatos, R. Alur, and V. Liu. Causalmesh: A causal cache for stateful serverless computing. *Proceedings of the VLDB Endowment*, 18(1):40–53, 2024.